Minería de Datos - Libro 1 - Clase 1 a 10

6. Clustering

Clase N° 6: Clustering en Minería de Datos

Introducción

Objetivo de la Clase

El clustering es una de las técnicas más utilizadas en minería de datos para descubrir patrones ocultos en grandes volúmenes de información. A diferencia del aprendizaje supervisado, el clustering es una técnica de **aprendizaje no supervisado**, lo que significa que no se cuenta con etiquetas previas para los datos.

En esta clase, aprenderemos a:

- Entender qué es el clustering y cuándo se aplica.
- Explorar diferentes algoritmos de clustering como K-Means, DBScan y Mapas Autoorganizados (SOM).
- Comparar los distintos enfoques y evaluar su desempeño.
- Implementar algoritmos de clustering en Python con Scikit-learn y otras bibliotecas.
- Analizar resultados y seleccionar el mejor modelo para un conjunto de datos.

Importancia del Clustering en Minería de Datos

- Segmentación de clientes en marketing para mejorar estrategias de ventas.
- Agrupación de documentos y textos en sistemas de recomendación.
- Detección de anomalías en transacciones bancarias o ciberseguridad.
- Clasificación automática de imágenes y datos biomédicos.

Ejemplo real: Un banco desea identificar diferentes perfiles de clientes según su comportamiento financiero. Mediante clustering, puede agrupar a los clientes en segmentos similares (clientes con bajo riesgo, clientes con alto consumo de crédito, etc.), lo que permitirá diseñar estrategias personalizadas.

Sección 1: Conceptos Fundamentales del Clustering

¿Qué es el Clustering?
El clustering es un proceso de agrupamiento de datos en subconjuntos homogéneos, de manera que los datos dentro de un grupo son más similares entre sí que con los datos de otros grupos.

1.1. Características del Clustering

- **No supervisado:** No requiere etiquetas previas en los datos.
- Identifica patrones ocultos en los datos sin intervención humana.
- Los resultados dependen del algoritmo elegido y de los parámetros utilizados.

1.2. Tipos de Clustering

Tipo de Clustering	Descripción	Ejemplo de Uso
Particional	Divide los datos en un número fijo de grupos.	K-Means, K-Medoids
Jerárquico	Crea una estructura de árbol para los clusters.	Dendrogramas en biología
Basado en densidad	Agrupa datos en regiones de alta densidad.	DBScan para detección de anomalías
Basado en redes neuronales	Utiliza modelos autoorganizados.	SOM en análisis de imágenes

¿Cómo elegir un tipo de clustering?

- Si se conoce el número de grupos → **K-Means o K-Medoids**.
- Si hay ruido o valores atípicos en los datos → **DBScan**.
- Si se quiere entender jerarquías en los datos → **Clustering jerárquico**.
- Si se necesita aprender representaciones complejas → **SOM (Self-Organizing Maps)**.

1.3. Ejemplo en Python – Creación de un Conjunto de Datos Sintético

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generar dataset sintético con 3 clusters
X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

# Graficar los datos sin etiquetas
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Datos sin etiquetar para Clustering")
plt.show()
```

Conclusión: Se observa que los datos tienen patrones naturales que pueden ser agrupados mediante clustering.

Ejercicio para la Sección 1

Objetivo: Comprender cómo se distribuyen los datos antes de aplicar clustering.

Instrucciones:

1. Genera un dataset con 4 grupos utilizando `make_blobs`.
2. Visualiza los datos en un gráfico de dispersión.
3. Reflexiona sobre qué técnica de clustering sería la más adecuada para estos datos.

Pista para la solución en Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# Generar dataset con 4 clusters
X, y = make_blobs(n_samples=400, centers=4, cluster_std=1.2, random_state=42)

# Visualizar los datos
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.title("Datos Sintéticos para Clustering")
plt.show()
```

Preguntas de Reflexión

1. ¿Cómo podríamos determinar automáticamente el número óptimo de clusters?
2. ¿Qué ventajas tiene visualizar los datos antes de aplicar clustering?
3. ¿Qué pasaría si los clusters estuvieran muy superpuestos?

Sección 2: Algoritmo K-Means

Objetivo: Comprender el funcionamiento del algoritmo K-Means, su proceso iterativo y cómo aplicarlo en un conjunto de datos.

2.1. ¿Qué es K-Means?

- K-Means es un algoritmo de clustering particional que divide los datos en K grupos basados en la minimización de la variación dentro de cada cluster.
- Se basa en la **distancia euclidiana** para asignar puntos a los clusters más cercanos.
- Es uno de los métodos más utilizados debido a su simplicidad y eficiencia computacional.

Aplicaciones de K-Means

- Segmentación de clientes en marketing.
- Agrupamiento de imágenes para clasificación.
- Compresión de datos en bases de datos.
- Análisis de comportamiento de usuarios en aplicaciones web.

2.2. Funcionamiento del Algoritmo

El proceso de K-Means sigue estos pasos:

1. Seleccionar el número de clusters K.
2. Inicializar los centroides de forma aleatoria.
3. Asignar cada punto al centroide más cercano.
4. Actualizar los centroides recalculando la media de cada cluster.
5. Repetir los pasos 3 y 4 hasta que los centroides no cambien significativamente.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generar datos sintéticos con 3 clusters
X, y = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)

# Aplicar K-Means con 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
y_kmeans = kmeans.fit_predict(X)

# Graficar clusters y centroides
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap="viridis", alpha=0.6)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c="red", marker="X", s=200, label="Centroides")
plt.title("Clustering con K-Means")
plt.legend()
plt.show()
```

Conclusión:

- Cada color representa un cluster asignado por K-Means.
- Los centroides en rojo indican los centros de cada grupo.

2.3. Elección del Número de Clusters K

Uno de los desafíos de K-Means es elegir el número óptimo de clusters. Los principales métodos son:

1. **Método del Codo (Elbow Method):** Calcula la suma de errores cuadrados para diferentes valores de K.
2. **Índice de Silhouette:** Mide la cohesión y separación de los clusters.

Ejemplo en Python – Método del Codo

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generar datos
X, y = make_blobs(n_samples=300, centers=5, cluster_std=1.0, random_state=42)

# Calcular la inercia para diferentes valores de K
inercia = []
valores_k = range(1, 10)
for k in valores_k:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inercia.append(kmeans.inertia_)

# Graficar el método del codo
plt.plot(valores_k, inercia, marker="o", linestyle="-")
plt.xlabel("Número de Clusters (K)")
plt.ylabel("Inercia")
plt.title("Método del Codo para Selección de K")
plt.show()
```

Conclusión: Se elige el punto donde la inercia deja de disminuir abruptamente, es decir, el "codo" de la curva.

Ejercicio para la Sección 2

Objetivo: Aplicar K-Means a un dataset real y determinar el número óptimo de clusters.

Instrucciones:

1. Carga el dataset de pingüinos de Seaborn (`sns.load_dataset("penguins")`).
2. Selecciona variables numéricas (ejemplo: `bill_length_mm`, `bill_depth_mm`).
3. Usa el método del codo para determinar el mejor K.
4. Aplica K-Means y visualiza los clusters.
5. Explica qué insights puedes obtener de la segmentación.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Cargar dataset de pingüinos
df = sns.load_dataset("penguins").dropna()
X = df[["bill_length_mm", "bill_depth_mm"]]

# Método del Codo
inercia = []
valores_k = range(1, 10)
for k in valores_k:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    inercia.append(kmeans.inertia_)

plt.plot(valores_k, inercia, marker="o", linestyle="-")
plt.xlabel("Número de Clusters (K)")
plt.ylabel("Inercia")
plt.title("Método del Codo - Pingüinos")
plt.show()

# Aplicar K-Means con K óptimo
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(X)

sns.scatterplot(x=df["bill_length_mm"], y=df["bill_depth_mm"],
                hue=df["Cluster"], palette="viridis")
plt.title("Clustering en Pingüinos")
plt.show()
```

Preguntas de Reflexión

1. ¿Cómo afecta la selección de K a la segmentación de datos?
2. ¿Qué limitaciones tiene K-Means cuando los clusters no son esféricos?
3. ¿Cómo podríamos mejorar los resultados de K-Means en datasets complejos?

Sección 3: DBScan y Clustering Basado en Densidad

Objetivo: Comprender el funcionamiento del algoritmo DBScan, sus ventajas sobre K-Means y cómo aplicarlo en Python.

3.1. ¿Qué es DBScan?

DBScan (*Density-Based Spatial Clustering of Applications with Noise*) es un algoritmo de clustering basado en densidad que **no requiere especificar el número de clusters**. Identifica áreas de alta densidad separadas por regiones de baja densidad, lo que permite detectar clusters de formas arbitrarias y manejar outliers de manera más efectiva que K-Means.

Ventajas de DBScan

- No requiere definir K de antemano.
- Puede detectar clusters con formas irregulares.
- Maneja bien el ruido y outliers en los datos.

Desventajas de DBScan

- No funciona bien si las densidades de los clusters son muy diferentes.
- Puede ser lento en datasets grandes.

3.2. Funcionamiento del Algoritmo

El algoritmo DBScan se basa en dos parámetros clave:

- **ε (Epsilon):** Determina la distancia máxima entre puntos vecinos.
- **minPts (Mínimo de puntos por cluster):** Define cuántos puntos se requieren para formar un grupo denso.

Cómo funciona:

1. Se selecciona un punto y se busca cuántos vecinos tiene en un radio ε.
2. Si hay al menos minPts vecinos, el punto se convierte en "punto núcleo" y se expande el cluster.
3. Si el punto tiene menos vecinos, se marca como "punto de ruido" (outlier).
4. Se repite hasta que todos los puntos han sido visitados.

3.3. Ejemplo en Python – Aplicando DBScan

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

# Generar datos con forma de media luna (difícil para K-Means)
X, y = make_moons(n_samples=300, noise=0.05, random_state=42)

# Aplicar DBScan
dbscan = DBSCAN(eps=0.2, min_samples=5)
clusters = dbscan.fit_predict(X)

# Visualizar resultados
plt.scatter(X[:, 0], X[:, 1], c=clusters, cmap="viridis", alpha=0.7)
plt.title("Clustering con DBScan")
plt.show()
```

Conclusión:

- DBScan detecta formas de clusters más complejas que K-Means.
- Los puntos marcados como `-1` son outliers que no pertenecen a ningún cluster.

3.4. Comparación entre K-Means y DBScan

Característica	K-Means	DBScan
Tipo de Clustering	Basado en particiones	Basado en densidad
Forma de los Clusters	Esféricos	Arbitrarias
Sensibilidad a Outliers	Alta	Baja
Número de Clusters	Debe definirse antes (K)	Se detecta automáticamente
Uso Recomendado	Cuando los clusters son bien definidos	Cuando hay formas complejas y ruido

¿Cuándo usar DBScan?

- Si los clusters tienen formas no esféricas.
- Si hay valores atípicos en los datos.
- Si no sabemos de antemano el número de clusters.

Ejercicio para la Sección 3

Objetivo: Aplicar DBScan a un dataset real y comparar los resultados con K-Means.

Instrucciones:

1. Carga el dataset de flores Iris (`sns.load_dataset("iris")`).
2. Selecciona dos características (`sepal_length` y `sepal_width`).
3. Aplica DBScan y visualiza los clusters.
4. Aplica K-Means y compara los resultados.
5. Explica qué diferencias observas entre ambos métodos.

Pista para la solución en Python

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN

# Cargar dataset Iris
df = sns.load_dataset("iris")
X = df[["sepal_length", "sepal_width"]].values

# Aplicar K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster_KMeans"] = kmeans.fit_predict(X)

# Aplicar DBScan
dbscan = DBSCAN(eps=0.5, min_samples=5)
df["Cluster_DBScan"] = dbscan.fit_predict(X)

# Visualizar
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
sns.scatterplot(x=df["sepal_length"], y=df["sepal_width"],
                hue=df["Cluster_KMeans"], palette="viridis")
plt.title("Clustering con K-Means")

plt.subplot(1, 2, 2)
sns.scatterplot(x=df["sepal_length"], y=df["sepal_width"],
                hue=df["Cluster_DBScan"], palette="viridis")
plt.title("Clustering con DBScan")
plt.show()
```

Preguntas de Reflexión

1. ¿Cuántos clusters detectó DBScan en comparación con K-Means?
2. ¿DBScan detectó puntos como outliers? ¿Por qué?
3. ¿Qué ventajas ofrece DBScan en este caso?

Sección 4: Evaluación de la Calidad del Clustering

Objetivo: Aprender a medir la calidad de los clusters generados con diferentes algoritmos mediante métricas de evaluación.

4.1. ¿Por qué evaluar un Clustering?

A diferencia del aprendizaje supervisado, en clustering no tenemos etiquetas verdaderas, lo que hace difícil evaluar si los clusters son correctos. Existen métricas que miden la **cohesión interna** y la **separación entre clusters**.

Métricas más utilizadas en clustering

Métrica	Descripción	Cuándo Usarla
Inercia (SSE)	Mide la suma de distancias de cada punto a su centroide (menor es mejor).	Evaluar K-Means.
Coeficiente de Silhouette	Mide cuán similares son los puntos dentro de un cluster frente a otros (rango -1 a 1, mayor es mejor).	Comparar diferentes algoritmos de clustering.
Índice de Davies-Bouldin (DBI)	Evalúa la compactación de los clusters y la distancia entre ellos (menor es mejor).	Comparar calidad general del clustering.

4.2. Aplicando Métricas de Evaluación en Python

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Cargar dataset de iris
df = sns.load_dataset("iris")
X = df[["sepal_length", "sepal_width"]].values

# Aplicar K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster_KMeans"] = kmeans.fit_predict(X)

# Aplicar DBScan
dbscan = DBSCAN(eps=0.5, min_samples=5)
df["Cluster_DBScan"] = dbscan.fit_predict(X)

# Evaluar con Silhouette Score
silhouette_kmeans = silhouette_score(X, df["Cluster_KMeans"])
mask = df["Cluster_DBScan"] != -1
silhouette_dbscan = silhouette_score(X[mask], df["Cluster_DBScan"][mask])

# Evaluar con Davies-Bouldin Index
dbi_kmeans = davies_bouldin_score(X, df["Cluster_KMeans"])
dbi_dbscan = davies_bouldin_score(X[mask], df["Cluster_DBScan"][mask])

print(f"Silhouette - K-Means: {silhouette_kmeans:.2f}")
print(f"Silhouette - DBScan:  {silhouette_dbscan:.2f}")
print(f"Davies-Bouldin - K-Means: {dbi_kmeans:.2f}")
print(f"Davies-Bouldin - DBScan:  {dbi_dbscan:.2f}")
```

Conclusión:

- Un mayor coeficiente de Silhouette indica clusters bien formados.
- Un menor Davies-Bouldin Index significa clusters más compactos y separados.

4.3. Interpretación de Resultados

¿Qué hacer si la métrica indica un clustering deficiente?

- Probar otro algoritmo de clustering (si K-Means falla, intentar DBScan).
- Ajustar hiperparámetros (K en K-Means o ε en DBScan).
- Aplicar reducción de dimensionalidad (PCA) para mejorar la separación de clusters.

```
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score

# Probar K-Means con diferentes valores de K
valores_k = range(2, 7)
silhouette_scores = []
for k in valores_k:
    kmeans = KMeans(n_clusters=k, random_state=42)
    clusters = kmeans.fit_predict(X)
    silhouette_scores.append(silhouette_score(X, clusters))

# Graficar Silhouette Score vs. K
plt.plot(valores_k, silhouette_scores, marker="o")
plt.xlabel("Número de Clusters (K)")
plt.ylabel("Coeficiente de Silhouette")
plt.title("Evaluación de Clustering con Silhouette Score")
plt.show()
```

Conclusión:

- Un pico en la curva indica el número óptimo de clusters.
- Si no hay un claro máximo, puede ser útil probar otros algoritmos.

Ejercicio para la Sección 4

Objetivo: Comparar diferentes algoritmos de clustering con métricas de evaluación.

Instrucciones:

1. Carga el dataset de diamantes (`sns.load_dataset("diamonds")`).
2. Selecciona variables numéricas (`carat` y `price`).
3. Aplica K-Means y DBScan.
4. Calcula el Coeficiente de Silhouette y el Davies-Bouldin Index para ambos.
5. Explica cuál algoritmo obtuvo la mejor segmentación.

Pista para la solución en Python

```
import seaborn as sns
import numpy as np
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Cargar dataset de diamantes (muestra)
df = sns.load_dataset("diamonds")
X = df[["carat", "price"]].values[:1000]

# Aplicar K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
labels_km = kmeans.fit_predict(X)

# Aplicar DBScan
dbscan = DBSCAN(eps=5000, min_samples=5)
labels_db = dbscan.fit_predict(X)

# Métricas
mask = labels_db != -1
print(f"Silhouette - K-Means: {silhouette_score(X, labels_km):.2f}")
print(f"Silhouette - DBScan:  {silhouette_score(X[mask], labels_db[mask]):.2f}")
print(f"Davies-Bouldin - K-Means: {davies_bouldin_score(X, labels_km):.2f}")
print(f"Davies-Bouldin - DBScan:  {davies_bouldin_score(X[mask], labels_db[mask]):.2f}")
```

Preguntas de Reflexión

1. ¿Cuál algoritmo obtuvo el mejor coeficiente de Silhouette?
2. ¿El Davies-Bouldin Index confirma los resultados del Silhouette Score?
3. ¿En qué situaciones puede fallar cada métrica?

Sección 5: Ejercicio Integrador Obligatorio

Entregar en la Tarea asignada en el campus. La misma se evaluará según la rúbrica publicada en la tarea.

Objetivo: Aplicar todas las técnicas de clustering vistas en la clase y compartir los resultados con la comunidad para discutir diferentes enfoques.

5.1. Ejercicio Integrador

Contexto: Supongamos que trabajamos en el departamento de análisis de datos de una empresa de e-commerce y necesitamos identificar grupos de clientes con patrones de compra similares para diseñar estrategias de marketing personalizadas.

Dataset a utilizar: UCI Online Retail Dataset
Contiene transacciones de una tienda online británica entre 2010 y 2011.

Variables principales:

Variable	Descripción
`InvoiceNo`	Número de factura
`StockCode`	Código del producto
`Description`	Descripción del producto
`Quantity`	Cantidad comprada
`InvoiceDate`	Fecha de la factura
`UnitPrice`	Precio unitario
`CustomerID`	Identificador del cliente

Variable	Descripción
`Country`	País del cliente

Tareas a realizar

1. **Preprocesamiento de Datos**
   - Eliminar valores nulos y registros con cantidades negativas.
   - Agrupar por `CustomerID` para obtener métricas por cliente: TotalCompras, CantidadTotal, GastoTotal y Frecuencia.
2. **Codificación y Normalización**
   - Convertir Frecuencia en una variable ordinal (Alta=2, Media=1, Baja=0, definida por terciles).
   - Normalizar CantidadTotal y GastoTotal usando MinMaxScaler o StandardScaler.
3. **Aplicación de Algoritmos de Clustering**
   - Aplicar K-Means y determinar el número óptimo de clusters con el método del codo.
   - Aplicar DBScan ajustando `eps` y `min_samples` para detectar clientes "outliers".
4. **Evaluación del Clustering**
   - Comparar resultados con Silhouette Score y Davies-Bouldin Index.
   - Justificar cuál método produce mejor segmentación de clientes y por qué.
5. **Interacción con una Herramienta de IA (LLM)**
   - Consulta sugerida: "*Tengo un dataset de transacciones de clientes de e-commerce. ¿Qué técnicas de clustering me recomendarías para segmentarlos?*"
   - Comparar la respuesta del LLM con lo aprendido en clase y reflexionar sobre su utilidad.

Reflexión Final de la Clase

- ¿Cuál algoritmo de clustering fue más efectivo en este ejercicio?
- ¿Cómo podríamos mejorar la precisión de la segmentación?
- ¿Qué limitaciones tiene el clustering en la minería de datos?

Bibliografía

1. Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Elsevier.
2. McKinney, W. (2017). *Python for Data Analysis*. O'Reilly Media.
3. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning*. Springer.
5. Kaufman, L., & Rousseeuw, P. J. (2005). *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley.
6. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer.
7. Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.
8. Seaborn Documentation. (n.d.). Recuperado de https://seaborn.pydata.org/
9. Scikit-learn Developers. (n.d.). Recuperado de https://scikit-learn.org/